

Multi-Agent Orchestration

When one agent isn't enough — the patterns,
frameworks, and protocols for teams of agents.

Session 3 of 3 • From Teams of One to Teams of Many



THE STORY SO FAR

Three sessions, one arc — and today the team grows.

SESSION 1

RAG

Give the LLM an open-book exam — retrieve, then generate.

SESSION 2

Agentic RAG

One agent decides how to use the book — rewrite, rerank, self-check.

SESSION 3 · TODAY

Multi-Agent

A team of specialists — each with their own books and tools, coordinated.

Complexity goes up. So does the need for engineering discipline — that's the through-line of all three days.

BY END OF SESSION

What you'll be able to do.

01 Explain why multi-agent exists

...and, just as important, when it doesn't help.

02 Name and draw the six patterns

Pipeline, routing, parallel, supervisor, hierarchical, evaluator.

03 Compare the major frameworks

OpenAI Agents SDK · LangGraph · CrewAI · Google ADK.

04 Explain MCP and A2A

How agents reach tools, and how agents reach each other.

05 Design an architecture

For a real problem — or argue why single-agent is better.

06 Spot the production pitfalls

Cost, error cascade, debugging, security, state, evaluation.

You'll leave with a design vocabulary — not just a tool list.

Morning · 9:00–12:00 (left) · Afternoon · 1:00–4:00 (right).

01	Block 1 — Why multi-agent	9:00	02	Shipping now — real world, compressed	1:00
03	Block 2 — The six patterns	9:20	04	When NOT to + what breaks	1:15
05	 Live demo	9:50	06	 Meet CrewAI + your task	1:30
07	 Activity 1 — Human Swarm	10:05	08	 Build your crew (hands-on)	1:45
09	 Break	10:25	10	 Demos from the floor	2:45
11	 Agent Spec Card	10:35	12	You have a hammer + my demo	3:15
13	Block 3 — Framework landscape	10:40	14	Forecast + wrap	3:25
15	Block 4 — MCP + A2A	11:05	16	 Admin — course close-out	3:30

BLOCK 1

BLOCK 1 OF 8 · WHY MULTI-AGENT

One agent has a ceiling.

A task that breaks one agent.

- The task: *"Research 5 competitors, analyse their pricing, write a comparison report, and build a slide deck."*
- One agent tries to do all of it: the context window fills up, tool selection degrades, output is mediocre at every step.
- It's not that the model is weak — it's that one set of instructions and one context window can only hold so much.

| *One agent, one prompt, one context window — there's a ceiling. The question is what to do when you hit it.*

LLM chat window, all four jobs. Guess what happens?

I'll paste the exact task from the last slide into a few plain chat windows — Claude / ChatGPT / Gemini, nothing special. Before I hit enter, commit to a prediction:

- **The task:** For Singapore's bubble-tea market: research the 5 biggest chains and their prices, say who's best value, write a 300-word comparison, and outline a 6-slide deck — all now, in one response.
- **Hands up:** who thinks the chat windows will *nail all four*?
- **Other hand:** who thinks it *invents the prices*, or goes shallow on at least one?

| Lock in your guess. Then watch closely when we run it — live.

It wasn't a bare LLM model. It was an agent.

Here's what actually happened — and why it frames the whole day:

- **The result:** it doesn't guess — it *searches the live web*, pulls real 2026 prices, catches a chain that just closed, and delivers all four.
- **The twist:** that box was **never a bare model**. Behind the cursor it's an *agent* — planning, calling a search tool, looping, composing. Tools + a loop, not raw text prediction. *That's* why it worked.
- **One level up:** *Research / Deep Research* modes spawn a **team of sub-agents** searching in parallel, reporting to a lead. That's **multi-agent orchestration**, hidden behind the text box.
- **So why learn this?** They built it into *their* chat. Over **your** data, **your** tools, **your** product — *you're* the one who orchestrates.

| *Multi-agent orchestration is already productised. Today: how it's built — and when not to.*

Why single agents hit a wall.

01 Context overload

Deep expertise in many domains overwhelms one prompt.

02 Tool overload

20+ tools confuse routing decisions; error rates climb.

03 Cognitive scatter

Research AND analyse AND write AND critique → mediocre at each.

04 Speed

Sequential tool use is slow; parallel agents work at the same time.

05 Specialisation

Different subtasks want different prompts, temperatures, even models.

Your agent from previous lectures had 3–4 tools. Imagine 20 — at some point it spends more time choosing tools than using them.

From one agent's tools to a team of agents.

SESSION 2 — AGENTIC RAG

One agent · → multiple tools · → loop until done · The agent decides which **tool** to call.

SESSION 3 — MULTI-AGENT

Multiple agents · → each with their own tools · → coordinated by an orchestrator · *An orchestrator decides which agent to call.*

🎬 VIDEO CLIP 1 · ~4 MIN + FRAMING

Multi-agent, in context.



Andrew Ng — "What's next for AI agentic workflows" (Sequoia, March 2024)

<https://www.youtube.com/watch?v=sal78ACtGTc>

Start at **9:20** · play ~4 min (stop around 10:52)

Ng identifies four agent *capabilities* — reflection, tool use, planning, and multi-agent collaboration. That fourth one is what we're unpacking today. But Ng's four are about what an agent can *do*. What we're about to cover is different: the six patterns for *how* multiple agents coordinate — the **topology** of collaboration, not the capability of a single agent.

| *Agent capabilities (Ng) vs orchestration patterns (Block 2) — different dimensions.*

BLOCK 2

BLOCK 2 OF 8 · THE SIX PATTERNS

Six shapes of collaboration.



BLOCK 2 · THE MAP

Six patterns. Each solves a different coordination problem.

1

Sequential Pipeline

Fixed-order stages; output feeds the next.

2

Routing / Handoff

Classify once, hand control to a specialist.

3

Parallel Fan-out

Run independent subtasks at the same time.

4

Supervisor / Workers

A manager plans and delegates to workers.

5

Hierarchical

Supervisors of supervisors; nested teams.

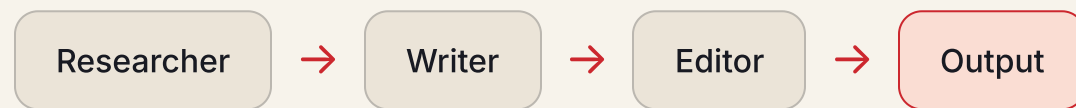
6

Evaluator-Optimizer

A generator and a critic loop to quality.

These aren't competing options. Most production systems combine several — think LEGO bricks, not a menu.

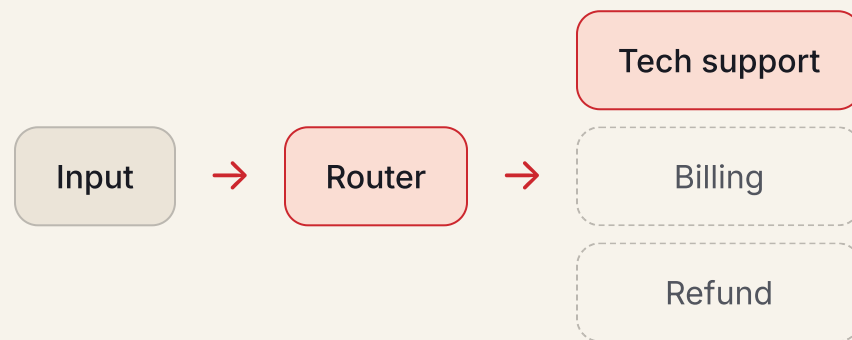
Sequential Pipeline.



- Output of one agent feeds the next, in a fixed order.
- Example: **Researcher** finds info → **Writer** drafts → **Editor** polishes.
- When to use: tasks with clear, non-overlapping stages.
- Callback: this is **prompt chaining** in Anthropic's terminology.

The simplest multi-agent pattern. Use when stages are well-defined and sequential.

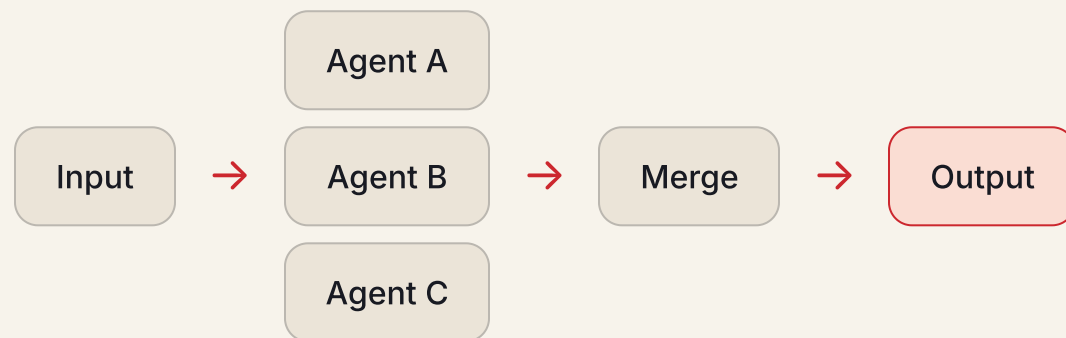
Routing / Handoff.



- A classifier agent examines the input and hands full control to the right specialist.
- Example: customer-service triage — **billing** vs. **technical** vs. **refund**.
- The receiving agent takes over the conversation entirely.
- OpenAI Agents SDK made this a first-class primitive: `handoff()`.

| *The gatekeeper pattern. Route once, specialise deeply.*

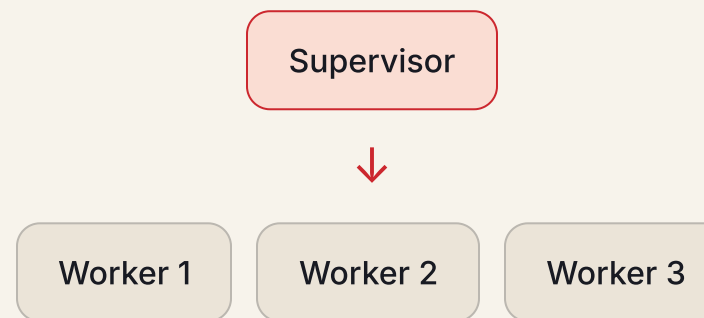
Parallel Fan-out / Fan-in.



- Multiple agents process subtasks at the same time.
- **Sectioning** — different subtasks in parallel (search DB A, B, C at once).
- **Voting** — run the same task several times, pick the best or the majority.
- When to use: independent subtasks, or redundancy for reliability.

| *Trade tokens for speed. Parallelise what's independent.*

Supervisor / Orchestrator-Workers.



- A manager agent plans subtasks, delegates to workers, then synthesises the results.
- It decides dynamically: which worker, what task, when to stop.
- Example: a coding agent that plans a change, then dispatches file-specific implementers.
- This is what **Claude Code** does — a main agent spawns sub-agents for investigation, implementation, review.

| *The workhorse of production multi-agent. One brain, many hands — and the most common pattern.*

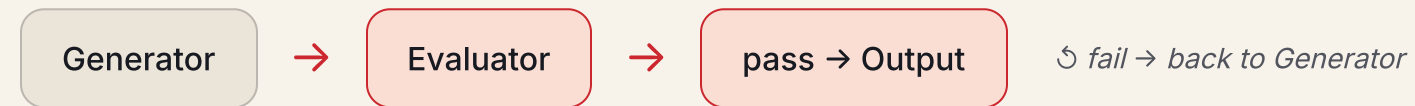
Hierarchical (Nested Teams).



- Supervisors delegate to sub-supervisors, creating tree structures.
- When to use: large, complex agent organisations with clear reporting lines.
- Trade-off: coordination overhead grows with depth.
- LangGraph supports this via nested subgraphs — but most production systems use **flat** supervisor-worker.

| *For genuinely large systems. Don't go hierarchical until you've proven you need it.*

Evaluator-Optimizer.

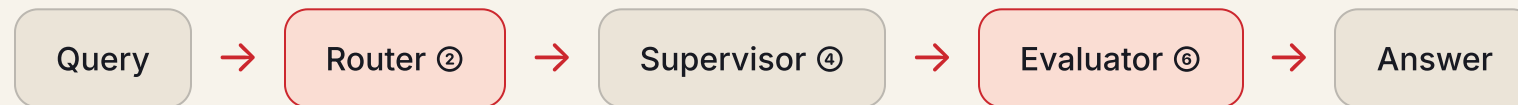


- One agent generates, another critiques — iterating until a quality threshold is met.
- Example: a code-writing agent + a test-running critic. Tests fail → the generator rewrites.
- Also used for: content refinement, translation quality, compliance checking.
- Callback: this is Session 1's **self-reflection**, now split across two agents with different prompts.

| *Separate the creator from the critic. Different prompts, different standards.*

BLOCK 2 · THE REAL SHAPE

Production systems are a composition,
not one pattern.



- Router → either an FAQ agent or a supervisor that fans out to a researcher + analyst → evaluator gate → answer.
- Four patterns, one system: routing ② + parallel ③ + supervisor ④ + evaluator ⑥.

| *Most real systems are LEGO compositions of the six — not a single clean pattern.*

● QUICK CHECK · SHOW OF HANDS

Map the problem to the pattern.

- 1 · *"A system that translates a document into 5 languages simultaneously?"*
- 2 · *"A chatbot that sends billing questions to one agent and tech questions to another?"*
- 3 · *"An agent writes code, another runs the tests and sends it back if they fail?"*

| *Call it out — every problem decomposes into one or more of the six.*

Six problems — watch which topology each one calls for.

#	PATTERN · WORKFLOW	DEMO PROMPT	WHAT TO WATCH
1	Sequential Pipeline — Researcher → Writer → Editor	Singapore's e-payment landscape	Three stages fire in fixed order; output improves at each.
2	Routing — Triage → HDB / CPF / IRAS	When can I sell my BTO flat? ... SRS tax relief?	Only ONE specialist activates per query.
3	Parallel — Announce → ZH / MS / TA → Aggregator	MRT closes early 11pm Sunday, N-S line maintenance	Three translators at once; wall-clock = one, not three.
4	Supervisor — Supervisor ⇌ Attractions / Food / Logistics	3 days in SG, family of 4, ~S\$1000, culture + street food	Supervisor dynamically picks the next worker.
5	Hierarchical — CEO → 2 Leads → 4 workers → Integrator	App for hawkers: inventory + pre-orders	Three tiers; longest run (9 agents) — coordination tax.
6	Evaluator-Optimizer — Generator ⇌ Evaluator (max 3)	Tagline for a 24-hr cold steel water bottle	Must be exactly 13 words; fails round 1, passes by 2-3.

Watch the topology, not the text — for each problem, notice why its shape fits the pattern it runs on: who talks to whom, and who decides.



Six problems, six patterns — each shape fit its task.

- Market brief flowed Researcher → Writer → Editor in a fixed line → **Pattern 1, Sequential.**
- Gov-services triage sent each question to exactly one specialist → **Pattern 2, Routing.**
- Three translators ran at once, an Aggregator merged them → **Pattern 3, Parallel.**
- Trip-planner Supervisor chose which worker to call and when to stop → **Pattern 4, Supervisor.**
- App-builder: CEO → Leads → workers, three tiers → **Pattern 5, Hierarchical.**
- Tagline Generator revised until the Evaluator approved → **Pattern 6, Evaluator-Optimizer.**

| *Each problem's structure pointed to its pattern — that's map-problem-to-pattern, live. And 4 & 5 were slowest and priciest: the complexity tax.*

Become a multi-agent system.

ROLE	CAN	CANNOT
Orchestrator	Read task, delegate by note, synthesise the final answer	Search, calculate, write, or critique
Researcher	Search the data table for facts & numbers	Calculate, write, or critique
Analyst	Calculate — profit margins, comparisons	Search, write, or critique
Writer	Compose the 3-sentence summary from given facts	Search, calculate, or critique
Critic	Approve / reject the summary with a written reason	Search, calculate, or write

Rules: all communication via written notes to the Orchestrator — no agent-to-agent talk. Task: from the FreshMart Q4 data, which product had the highest profit margin? Write a 3-sentence summary. 2 min setup • 10 min • 8 min debrief.

Q4 2025 Product Performance — FreshMart Pte Ltd

PRODUCT	REVENUE (S\$)	COST OF GOODS SOLD (S\$)	MARKETING SPEND (S\$)	UNITS SOLD
Organic Granola	184,000	96,000	22,000	4,600
Cold-Pressed Juice	312,000	205,000	35,000	7,800
Protein Bars	147,000	58,000	31,000	5,880
Kombucha	223,000	134,000	18,000	6,360
Acai Bowls	95,000	61,000	14,000	1,900

Task: which of these 5 products had the highest profit margin last quarter? Write a 3-sentence executive summary with the exact numbers.

What did you just notice?

- **Where was the bottleneck?** Usually the Orchestrator — a single point of failure.
- **What happened when the Researcher returned wrong data?** Error propagation.
- **Did the Critic reject anything?** That's the evaluator-optimizer loop, live.
- **Could any agents have worked in parallel?** A fan-out opportunity you felt by hand.

| *You just WERE a multi-agent system. Every friction you felt is a real engineering problem — we'll name them in Block 7.*

INTERMISSION

10_{min}

Back soon. Resume at the time on screen.

• ✎ MINI-EXERCISE • 5 MIN • POST-BREAK WARM-UP

Write an Agent Spec Card.

Describe one agent — the role you played in the Human Swarm, or invent a new one. On one notecard:

- **Role** — one sentence.
- **Tools** — what it can access.
- **Inputs** — what it receives.
- **Outputs** — what it produces.
- **Guardrail** — what it must NOT do.

| *Then pair up: combine your two agents into a mini-system. What integration problem do you immediately see? That's the first problem every framework is trying to solve.*

BLOCK 3

BLOCK 3 OF 8 · THE FRAMEWORK LANDSCAPE

Frameworks handle the plumbing.

Why frameworks exist.

- You could build orchestration from scratch — it's just LLM calls and routing logic.
- Frameworks handle the plumbing: **agent lifecycle, message passing, state, tool execution, tracing.**
- The landscape is crowded and fast-moving. We'll cover the four that matter most right now.

| *Frameworks are a means, not an end. By the end of this block you'll know which fits which problem — you won't need all four.*

There's no winner — there's the right tool.

	AGENTS SDK	LANGGRAPH	CREWAI	GOOGLE ADK
Released	Mar 2025	2024	2024	Apr 2025
Philosophy	Minimal	Max control	Role-based	Interoperable
Core metaphor	Handoffs	Graph	Crew	Workflows + A2A
Learning curve	Low	High	Medium	Medium
Maturity	Good	Highest	Good	Maturing
Lock-in risk	OpenAI-leaning	LangChain	Standalone	Google-leaning
Best for	Routing/handoff	Stateful flows	Prototyping	Cross-framework

BLOCK 3 · HOW TO CHOOSE

The flowchart you'll use on Monday morning.

IF YOU'RE...	USE
Primarily routing a request to the right specialist	OpenAI Agents SDK
Needing complex stateful workflows with human-in-the-loop	LangGraph
Best described as a team of roles collaborating	CrewAI
Needing agents from different vendors to interoperate	Google ADK
Not sure you need multi-agent at all	Start with one agent

| *The default is single-agent. Only move up when you've hit its ceiling.*

Anthropic doesn't ship a multi-agent framework.

- Instead: an influential guide (*Building Effective Agents*), the Claude Agent SDK, and MCP.
- Claude Code uses a single-agent-with-sub-agents model — it spawns scoped sub-agents as needed.
- Core philosophy: *"Find the simplest solution possible, and only increase complexity when needed."*
- Start with one agent. Add a second only when you can prove it helps.

| *The most important framework advice in the room: maybe you don't need one yet. And it's often right.*

What does multi-agent code actually look like?

```
# CrewAI – role metaphor
researcher = Agent(role="Researcher", goal="Find market data", tools=[search])
analyst    = Agent(role="Analyst",    goal="Identify trends",  tools=[calc])
writer     = Agent(role="Writer",     goal="Draft the report", tools=[])
crew       = Crew(agents=[researcher, analyst, writer], process="sequential")
result     = crew.kickoff(task="Q3 analysis")

# OpenAI Agents SDK – handoff metaphor
triage = Agent(name="Triage", instructions="Route to a specialist")
billing = Agent(name="Billing", instructions="Handle billing")
tech    = Agent(name="Tech",    instructions="Handle tech issues")
triage.handoffs = [billing, tech]
run(triage, "I can't log in") # → tech
```

Don't memorise this. Defining an agent is ~3 lines — the complexity lives in the design, not the syntax.

Also worth knowing.

- **AutoGen (Microsoft)** — pioneered multi-agent conversations; now in maintenance mode. It merged with Semantic Kernel into the **Microsoft Agent Framework** (1.0 GA, Apr 2026) — that's where new Microsoft work goes.
- **Amazon Bedrock Agents** — managed multi-agent on AWS. Good if you're already on AWS.
- **Smolagents (HuggingFace)** — lightweight, code-agent focused.
- **PydanticAI** — type-safe for teams that love Pydantic. **Mastra** — TypeScript-native.

| *The landscape moves fast. Focus on the big four + the Anthropic contrarian view.*

BLOCK 4

BLOCK 4 OF 8 · MCP & A2A

How agents reach tools — and each other.

BLOCK 4 · THE PLUMBING

Two interoperability problems, two protocols.

AGENT ↔ AGENT

A2A

Agent ↔ Agent ↔ Agent. How agents reach each other.

AGENT ↔ TOOLS / DATA

MCP

Agent → Database · API · File system. How agents reach tools.

MCP = USB-C for AI. A2A = HTTP for agents. Complementary, not competing.

MCP — the Model Context Protocol.

- Created by Anthropic (2024). An open standard for connecting AI apps to external data and tools.
- Think **USB-C for AI**: one interface, any app to any tool. Servers expose tools/data; clients (AI apps) discover and call them.
- Concretely: point an app at a **filesystem** server → it reads your files; a **GitHub** server → it opens PRs. Same protocol, any tool.
- The one that won: first-class support in Claude, ChatGPT, Gemini, Copilot, Cursor, VS Code — OpenAI and Google both adopted it in 2025. 10,000+ servers published.
- **Status**: the de-facto standard — now stewarded by the Linux Foundation (Agentic AI Foundation, Dec 2025). Production-ready and stable.

| *A solved problem. If you build tools for agents, build them as MCP servers — framework- and model-agnostic.*

A2A — agents reaching other agents.

- Created by Google (2025), donated to the Linux Foundation. The open standard for agent-to-agent collaboration — **complementary to MCP** (MCP reaches tools; A2A reaches agents).
- Two ideas worth knowing: **Agent Cards** — JSON advertising what an agent can do (discovery) — and **Tasks** — a unit of work one agent delegates to another. Transport: JSON-RPC over HTTPS.
- **v1.0 shipped (Mar 2026)** — the first stable, production-ready spec. 150+ organisations; an 8-seat steering group (Google, Microsoft, AWS, Cisco, Salesforce, ServiceNow, SAP, IBM); SDKs in 6 languages.
- Landing in the major clouds (Azure AI Foundry, AWS Bedrock AgentCore) with early enterprise production use — but still **younger and less pervasive than MCP**.

| A year ago: 'solid spec, wait and see.' By 2026 it's v1.0 and in the major clouds — real, if younger than MCP. Know both: MCP for tools, A2A for agents.

Where the industry is heading.

- Agents from different frameworks (ADK · LangGraph · CrewAI) talk to each other over **A2A**.
- Each agent reaches tools and data (DB · API · Files) over **MCP**.
- An orchestrator + tracing spans the whole system.

| *A standards-based ecosystem where agents are interchangeable components. MCP + A2A = the plumbing of that future.*

INTERMISSION

60_{min}

Back soon. Resume at 1:00 PM sharp.

B L O C K

AFTERNOON · 30 MIN WHAT'S REAL & WHAT BREAKS → 15 MIN MEET CREWAI → 60 MIN BUILD → DEMOS → WRAP

This afternoon, you build one.

It's already shipping — and it's simpler than the hype.

- **Enterprise:** Salesforce AgentForce · ServiceNow · Microsoft Copilot Studio · Amazon Bedrock — routing / supervisor with a handful of specialists.
- **The tools you code with:** Claude Code, Copilot Workspace, Devin — planner → worker → reviewer, under the hood.
- **Singapore, running today:** GovTech PAIR · DBS & OCBC (fraud, a 30,000-employee assistant) · Grab (allocation, fraud, pricing).
- **The through-line:** almost every real system is **2–5 agents** and simple patterns — not swarms of fifty.

| *The hype says swarms. Production says two-to-five specialists. Hold that thought — in an hour you'll build one yourself.*

*“Find the simplest solution possible,
and only increase complexity when
needed.”*

— ANTHROPIC · BUILDING EFFECTIVE AGENTS

Every agent you add multiplies cost.

	SINGLE	3 AGENTS	5 AGENTS
LLM calls / request	1–3	5–10	10–20+
Latency	Low	Medium	High
Cost / request	\$	\$\$–\$\$\$	\$\$\$–\$\$\$\$
Debugging	Simple	Moderate	Hard
Failure surface	Small	Medium	Large

Production teams report 3–10× cost vs single-agent. Complexity is a cost — pay it only when the return is worth it.

When to go multi-agent — and when not to.

✅ GO MULTI-AGENT WHEN

The task needs genuinely different expertise domains · Subtasks parallelise for a meaningful speed gain · You need separation of concerns for security (PII boundary) · Quality requires a separate critic/evaluator · Single-agent has been tried and **measurably failed**

❌ STAY SINGLE-AGENT WHEN

One well-prompted agent with good tools can handle it · Latency budget is tight (< 1–2 seconds) · The corpus is small and structured · You're optimising for cost · You haven't proven the single-agent ceiling yet

Six hard problems. Different fixes for each.

1

Error cascade

Mistakes propagate and amplify across agents.

2

Cost explosion

Calls multiply with agents, turns, and retries.

3

Debugging complexity

Which agent decided wrong, with what context?

4

Security & trust

Every boundary is an attack surface.

5

State management

Who owns shared, mutable state?

6

Evaluation

Score per-agent AND the whole system.

You'll meet #1 (cascade) and #2 (cost) live in your own crew within the hour. We'll deep-dive one of these at the close.

B L O C K

🔧 WORKSHOP · 15 MIN INTRO · 60 MIN BUILD

Meet CrewAI — then build.

Four nouns. That's almost all of CrewAI.

THE WHO

Agent

A persona: role + goal + backstory. Your Researcher, Analyst, Writer.

THE WHAT

Task

A unit of work with an expected_output, assigned to one agent.

THE TEAM

Crew

Agents + tasks, assembled and kicked off with one topic in.

THE HOW

Process

sequential = pipeline · hierarchical = supervisor. Two patterns, no wiring.

Two of the six patterns are one word each: Process.sequential (Pattern 1) and Process.hierarchical (Pattern 4). That's why it's a great place to START.

The whole crew is about a dozen lines.

```
from crewai import Agent, Task, Crew, Process

researcher = Agent(role="Researcher", goal="Find 3 solid facts on {topic}", backstory="Meticulous; loves correct, simple facts.", tools=[search])
analyst     = Agent(role="Analyst",    goal="Find the one angle that matters", backstory="...")
writer      = Agent(role="Writer",     goal="Write a one-page brief",      backstory="...")

research = Task(description="List 3 facts on {topic}", expected_output="3 bullets", agent=researcher)
angle     = Task(description="Pick the key angle", expected_output="1 paragraph", agent=analyst, context=[research])
brief     = Task(description="Write the brief", expected_output="~250 words", agent=writer, context=[research, angle])

crew = Crew(agents=[researcher, analyst, writer], tasks=[research, angle, brief], process=Process.sequential, verbose=True)
result = crew.kickoff(inputs={"topic": "cashless payments in Singapore"})
```

Defining an agent is ~3 lines; wiring the pipeline uses `context=`. The design is the work, not the syntax. You'll steer your AI assistant to write this — then read it and tweak.

Your task: turn a topic into a one-page brief.



- **Pick a domain you actually care about** — your industry, a Singapore scenario from this morning, or your current work problem.
- **Researcher** gathers the facts (give it a search tool) → **Analyst** finds the angle that matters → **Writer** produces the brief.
- Give each a **sharp role and goal**, and each task a **specific `expected\_output`** — vague specs are where crews go mediocre.
- Success = one topic in, a finished one-page brief out, produced by three agents that hand work to each other.

| *Same skeleton for everyone; your topic and roles make it yours. That's what makes the demos worth watching.*

Steer your AI assistant through four steps.

01 Scaffold a working crew

CrewAI CLI (`crewai create crew`) or ask your AI assistant (e.g. 'Create a very simple example to demonstrate the CrewAI agent framework.') — start from something that already runs.

02 Define three agents

Researcher (+ search tool), Analyst, Writer. Clear role, goal, backstory for each.

03 Define three tasks

One per agent. Specific `expected_output`. Wire hand-offs with `context=[...]`.

04 kickoff(topic)

Pass your topic as an input. Watch the verbose log. Read the brief. Tweak. Re-run.

Don't hand-type. Describe the intent → read what it writes → tweak. The verbose log IS your trace — the thing we said you can't debug without.

•  PREFLIGHT · 2 MIN — HANDS UP IF ANY FAIL

Before you build.

Catch the failures now, not at minute 35. Raise a hand on any line that errors and we fix it together:

- **Coder workspace access** — able to login and open Terminal or VSCode.
- **Your LLM is reachable** — Ollama running with a model pulled, *or* an API key in your `.env` (e.g. 'Can you inspect the environment variables, and find any available LLM key or token?').
- **Your AI assistant is open** — coding assistant running, a blank workspace in front of you.

| *If your crew runs later, help a neighbour whose doesn't.*

• 🛠 BUILD • 60 MINUTES • YOU + YOUR AI ASSISTANT

Build your crew.

Goal: one topic in → a one-page brief out, from three agents that hand work to each other. The four steps are on the last slide.

- **Steer, don't type** — describe each agent and task to your assistant; read what it writes; tweak the roles and expected_outputs.
- **Watch the trace** — the verbose log tells you who handed off to whom, and where it went wrong.
- **Stuck > 5 min?** — grab me or a neighbour. If your crew runs, help someone whose doesn't.
- **Comfort break** — take one whenever your crew is running; this hour is self-paced.

| *By half-way everyone should have a running sequential crew. Fast finishers: the next slide is your stretch.*

Fast finisher? Add a manager.

- A sequential crew runs the **same steps in the same order** every time — no matter the topic.
- Some jobs need a **coordinator**: decide what to do, delegate to the right specialist, and **reject weak work** and ask for a redo.
- That's **supervisor-worker** — Pattern 4 from the lecture. In CrewAI it's `Process.hierarchical`.
- Add **one Manager agent**, set `manager_agent=...`, change process to hierarchical. ⚠ Switch the process but forget the manager and it breaks — the classic bug.

Re-run and read the new trace: now it shows delegation and review, not a fixed line. Same three workers — you added a layer above them.

● CHECKPOINT · ~30 MIN IN

By now, every laptop should have...

- A crew of **three agents** that runs end to end.
- A **one-page brief** generated from the topic you chose.
- A **verbose log** you can point to and say "here's where the Analyst handed off."

| *If yours runs, help a neighbour whose doesn't. If it produces junk — hold that thought; that junk is the whole point of the wrap.*

• 🎬 DEMOS FROM THE FLOOR • 30 MIN • 4-5 CREWS

Show us your crew.

Volunteers — share your screen and walk us through:

- **The best brief** your crew produced — and the topic you chose.
- **A funny hallucination** — where did an agent confidently make something up?
- **Did your manager reject a worker** and ask for a redo? (stretch crews)
- **Which of the six patterns** did you actually end up using?

| *As each demo runs, I'll name the patterns and the failure modes live — error cascade, cost, the evaluator loop — the whole of Block 7, from your machines.*

You have a hammer. Most things aren't nails.

- Most of what you'll be tempted to build as a crew is **cheaper, faster, and more reliable** as a single agent or a plain chain.
- A crew earns its cost only when the work genuinely **splits across distinct specialists** with different tools or expertise.
- Every extra agent is more latency, more tokens, and **another thing that can fail silently**.
- The rule from this morning still governs: **start with one. Add agents only when you have evidence.**

You just felt it: your brief crew made ~10 calls where a single good agent might make one. Knowing when NOT to build a crew is the senior skill — measured from the other direction.

What stands between your crew and production.

- **Cost** — N agents $\times$ several calls per task; a hierarchical crew can be many times the price of one agent.
- **Latency** — it compounds; each hand-off waits on the one before it.
- **Non-determinism** — same input, different runs; the trace is your only friend when debugging.
- **The harness** — token budgets, retries, guardrails, scoped permissions, validation gates between agents — real work, none of it done today.

Your crew ran on localhost with no seatbelts. Fine for learning; not fine for anything real. In multi-agent, the harness grows faster than the agents.

One more — here's what it looks like when you keep going.

You built the demo. Here's something to inspire you:

- Same patterns you just used — now with **real tools (MCP)**, and **validation gates**.
- Watch where the harness lives: budget caps, retries, and the check between agents that stops a bad hand-off.
- Notice it's still **2–5 agents** — the discipline holds all the way up.

| *Same six patterns, one rung higher.*

Where this is heading.

PRODUCTION-READY NOW

Use it today

Single agent with tools · Supervisor + 2–4 specialists · Pipelines with validation gates · MCP for tools

PROMISING BUT EARLY

Watch closely

MA frameworks (debugging still hard) · A2A protocol — adoption coming · Google ADK 2.0 — young ecosystem

STILL HYPE

Not yet

Self-organising swarms of 10+ · Agent teams replacing human teams · Cross-org agent commerce

Honest about what works. Honest about what doesn't — yet.

TAKE IT FURTHER

Everything from today, in one place.

- **Anthropic** — "Building Effective Agents".
- **DeepLearning.AI** — Multi AI Agent Systems (CrewAI) · AI Agents in LangGraph · A2A (IBM).
- **GitHub** — openai/openai-agents-python · langchain-ai/langgraph · crewAIInc/crewAI · google/adk-python.
- **Protocols** — modelcontextprotocol.io · a2a-protocol.org. **Demos** — ChatDev · MetaGPT · AI Town.

| Scan the QR on the day — everything from today lives on that page.

Start simple.

Today you didn't just learn the patterns — you built a team of agents and felt its cost. Start simple. Measure everything. Add complexity only when it earns its place.